Open Evidential Tool Bus (O-ETB)
Development Version v1.1.0 (Prototype #1)
User Manual
May 2024

Contents

1. Version History 3
1.1 Overview of v1.1.0 3
1.1.1 O-ETB repository organisation 3
1.1.2 Functional configurations 4
1.2 Features foreseen in near-future versions 5
1.2.1 Assurance Server configuration 5
1.2.2 Import of MedSecurance Model information 5
1.2.3 Development of MedSecurance argument patterns and workflows 5
1.2.4 Development of Tool Agents for MedSecurance tools 5
1.3 Summary of earlier work leading to O-ETB 5
1.3.1 D-MILS Project M-BAC tool 6
1.3.2 CITADEL Project AM-ETB tool 6

2. The O-ETB Assurance Tool and Assurance Server 7
2.1 Enhanced 'etb' Assurance Tool 8
2.1.1 Command line options 8
2.1.2 ETB interactive commands 8
2.1.3 Timing Commands 11
2.1.4 Command procedures and scripts 11
2.2 Enhanced assurance case pattern specification language 11
2.2.1 Basic Syntax 11
2.2.2 Syntax of parameterized elements 12
2.3 Assurance Server 12
2.3.1 Server sessions 13
2.3.2 Server APIs 13

3. Installation and Operation 14
3.1 Overview 14
3.2 Prerequisites 14
3.3 Installing and Running the 'etb' assurance tool 14
3.3.1 Install needed software 15
3.3.2 Install the O-ETB source files and/or executable 15
3.3.3 Compile the 'etb' policy tool 15
3.3.4 Test the installed 'etb' tool 16
3.3.5 Running the examples 16
3.4 Installing and Running the 'etb-server' 16
3.4.1 Install SWI-Prolog 17
3.4.2 Install the O-ETB source files and/or etb-server executable 17
3.4.3 Initiating the 'etb-server' 17

3.4.4 Test the installed 'etb-server' *17*
3.5 O-ETB Tool & Server Deployment for MedSecurance 18

4. Principles of Operation 19
4.1 Argument patterns 19
4.2 Argument pattern instantiation 19
4.3 CASES repository 20
4.4 EVIDENCE repository 20

5. Annex A: Populating the O-ETB Knowledge Base 21
5.1 Model information 21
5.2 Argument Patterns 21
5.3 Workflow information 21

6. Annex B: O-ETB Testing 22
6.1 Volume and Load Tests 22
6.2 Performance and endurance tests 22
6.3 HTTP API correctness tests 23

Figures

Figure 1 O-ETB repository top-level structure 3
Figure 2 O-ETB architecture 7
Figure 3 Pattern instantiation rewrite rules 19

Tables

Table 1 General purpose ETB tool commands 8
Table 2 Data and control commands of the ETB tool 10
Table 3: Instantiation correctness self-tests 23
Table 4: O-ETB correctness HTTP API tests 24

1. VERSION HISTORY

This section presents a synopsis of versions of the Open Evidential Tool Bus (O-ETB) in reverse chronological order. The first subsection, which pertains to the current version, is followed by a preview of the anticipated next version. When a new version is released the preview subsection will be removed from the document and a new version subsection and new preview subsection will be added in the *front* of the then-current first subsection.

1.1 OVERVIEW OF V1.1.0

Version 1 of O-ETB is a continuing development derived from Adaptive MILS ETB v3 (AM-ETB), implemented by Université Grenoble Alpes as part of the CITADEL Project, in continuation of automated assurance case construction support implemented by University of York as part of the D-MILS Project. For more on this legacy work see Section 1.3.

O-ETB is intended to be an open source development that will be available on GitHub. O-ETB v1 and anticipated future adaptations and improvements are being developed under the auspices of the European Commission's MedSecurance Project. Please consult the MedSecurance design and specification documents (e.g. D2.2 Requirements & Design of Automated IoMT Security Analysis & Assurance Toolbox) for a requirements specification of the features foreseen for this development. Prototype #1 of O-ETB for the MedSecurance Project is designated as O-ETB v1.1.0.

1.1.1 O-ETB repository organisation

O-ETB adopts a new repository structure, consolidating source code and adding new directory structure elements for a dynamic knowledge base, repositories for changeable persistent data comprising assurance cases and assurance evidence, artefact collections comprising certification assurance packages, build space, and documents. Figure 1 shows the current top-level structure of the O-ETB repository.

FIGURE

Figure 1 O-ETB repository top-level structure

1.1.1.1 Source Code

The static Prolog source code has been consolidated into a "src" sub-directory of the source repository. Module files have been added for new functionalities such as command line interpreter, command-invoked actions, command procedures and scripts, self-tests, and parameterization of the implementation.

A significant organisational enhancement has been the addition of a global parameters file ("param.pl") that consolidates O-ETB operational settings and constant definitions in one place complemented with corresponding references to the parameters at appropriate places in the source code.

1.1.1.2 Knowledge Base

A distinct place in the runtime file structure has been created for the knowledge base used by O-ETB ("KB" directory). The knowledge base includes the library of assurance case argument patterns ("PATTERNS" sub-directory), the library of system models ("MODELS" sub-directory), a library of workflow patterns ("WORKFLOWS" sub-directory), and in the future will be extended with other directories for expert knowledge applicable to the construction of assurance cases and specifications for the requirements on the content and presentation for certification assurance packages.

1.1.1.3 Assurance Case and Evidence Repositories

Under the "REPOSITORY" directory the assurance case repository ("CASES" sub-directory) and the evidence repository ("EVIDENCE" sub-directory) are located. Separate directories are created under CASES for each assurance case. The EVIDENCE directory is organized by directories corresponding to evidence categories and under those are sub-directories are used for individual evidence records.

1.1.1.4 Certification Assurance Package

The runtime file structure includes a default location ("CAP" directory) for the export of collections of artefacts tailored to specific certification targets.

1.1.2 Functional configurations

This version of O-ETB is configured as an interactive *Assurance Tool* though it contains some new groundwork for an *Assurance Server* configuration. Numerous source files are shared to build both the Assurance Tool and the Assurance Server.

Functionally, the Assurance Tool has been extended with a new extensible command line interpreter for enhanced development, testing, and user interaction; and initial steps toward the background-running server that will enable concurrent use by multiple users and processes in the construction of separate assurance cases and certification assurance packages.

More information about the functional configurations mentioned here may be found in the main body of this User Manual and its Annexes.

1.1.2.1 Assurance Tool configuration

The Assurance Tool has been extended with a command line interpreter, including commands that provide access to internal assurance case and evidence functions (also to be accessed via external API invocations), knowledge base loading and inspection, and functions to facilitate ongoing development such as interactive tracing and testing.

The Assurance Tool may be invoked to operate on the centralized repository or on a working repository that has the same structure. This mode of use is

intended for development, testing, and/or independent assurance case and certification development efforts.

1.1.2.2 Assurance Server configuration

The source code for the current version includes extensions to initiate a server mode of background operation and the makefile contains a target for an Assurance Server configuration. However, the code does not yet include all server functionality nor an implementation of the proposed APIs.

1.2 FEATURES FORESEEN IN NEAR-FUTURE VERSIONS

1.2.1 Assurance Server configuration

A distinct "etb-server" executable is built to provide a distinct entry point to the O-ETB assurance server mode of operation. The server will run in the background and be interacted with by other processes through a RESTful API in addition to the current filesystem-based interaction protocols. This API will provide programmatic access to some functions in common with those of the command language. The Assurance Server is also discussed in Sections 1.1.2.2, 1.2.1, and 2.3.

New RESTful HTTP APIs for the assurance server have begun to be identified as part of the activities to develop O-ETB Prototype #1 but are not yet been implemented. The Prototype #2 will implement the basic server and at least the minimum subset of these identified APIs. The APis are also discussed in Section **Error! Reference source not found.**

1.2.2 Import of MedSecurance Model information

To be supplied. A preliminary study of the model information currently needed by O-ETB and how it might be provided by other MedSecurance modelling activities is provided in Annex A.

1.2.3 Development of MedSecurance argument patterns and workflows

To be supplied.

1.2.4 Development of Tool Agents for MedSecurance tools

To be supplied.

1.3 SUMMARY OF EARLIER WORK LEADING TO O-ETB

O-ETB represents an evolutionary development of the approach and implementation of assurance case automation support continuing the spirit and developments of the D-MILS (2012-2015) and CITADEL (2016-2019) EC projects. These previous assurance efforts were done in the context of the MILS architectural approach, encompassing platform, architecture and model-based development and verification methods for systems requiring high-assurance of security, safety and other key system properties. These efforts were supported

by a methodology and tool set supporting rigorous specification and verification of MILS systems.

MedSecurance requires a broadening of the scope of application of these methods and tools from a MILS platform and architecture environment to alternative and more generalized environment and a different and more diverse selection of assurance supporting tools.

1.3.1 D-MILS Project M-BAC tool

To be supplied.

1.3.2 CITADEL Project AM-ETB tool

To be supplied.

2. THE O-ETB ASSURANCE TOOL AND ASSURANCE SERVER

The O-ETB functional architecture is shown in Figure 2.

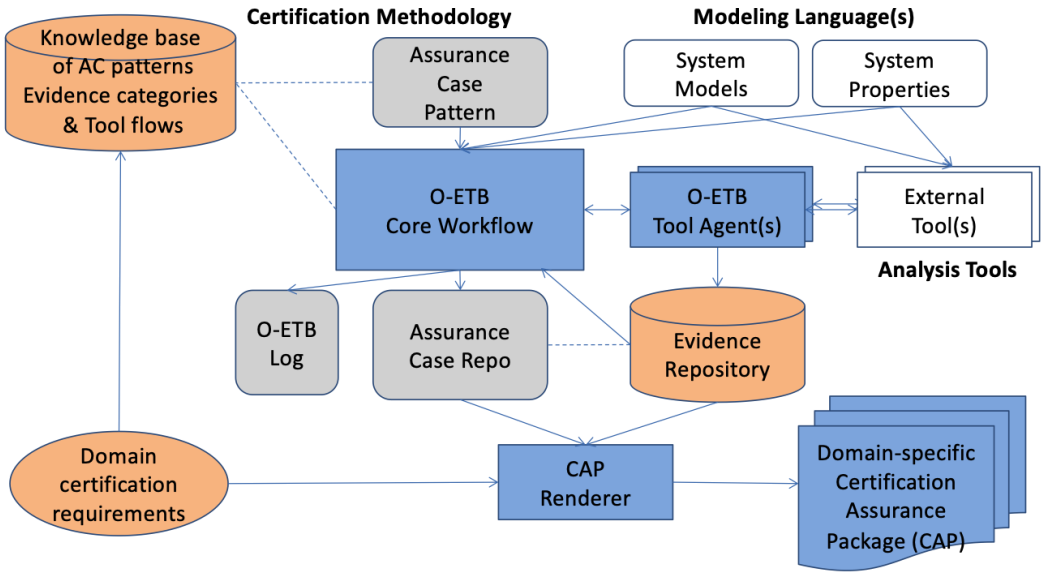

Figure 2 O-ETB architecture

The tool agents include calls to the O-ETB Server through the RESTful Evidence Management Interface and by interacting directly with files in the Evidence Repository.

2.1 ENHANCED 'ETB' ASSURANCE TOOL

The 'etb' tool enables standalone AC pattern development, AC pattern instantiation and tool development and testing. It is extended with the ability to start the etb server to run in the background. The etb tool provides the ability to work with a pattern as it is being created or modified, to test its instantiation algorithms and use of the knowledge base.

2.1.1 Command line options

When a compiled version of the O-ETB Assurance Tool is started from the command line as "etb", several command line options are recognized.

- `--token`, `-t` <epptoken> use the token to authenticate requests to the ETB
- `--port`, `-p` <portnumber> server should listen on specified port number
- `--selftest`, `-s` run self tests on startup
- `--verbose`, `-v` show all messages
- `--jsonresp`, `-j` encode responses in JSON

2.1.2 ETB interactive commands

The 'etb' Assurance Tool is a command driven application. After starting 'etb' it offers the prompt "etb>". There are basic general commands available in the normal mode (user) and an extended set of commands for use by a developer in development mode (advanced). The command interpreter is easily extensible. The command interpretation machinery and the generic commands are defined in the command module. Commands for specific groups of functionalities can be defined in separate smaller files where the syntax, argument types, operational semantics and help text for such commands can be isolated. Commands that invoke ETB-specific functions are currently defined in a separate file command_etb.pl. It is anticipated that a set of commands for knowledge base or evidence management may be defined separately in the future. Entering the command "help" will list all the commands available in the current mode.

Table 1 shows the general purpose commands available in the ETB tool.

Table 1 General purpose ETB tool commands

`advanced. | basic. | developer. | etb.`	Switch to advanced (basic, developer or etb) mode.
`halt.`	Exit the etb tool. (Will also terminate a server spawned from

	an interactive instance.)
`help.`	List the commands available in the current mode.
`help(` *<command name>* `).`	Give a synopsis of the named command.
`proc(` *<proc name>* `[, step] ).`	Run the named stored procedure, optionally pausing after each command.
`proc(` *<proc name>* `[, verbose] ).`	Run the named stored procedure, optionally verbose.
`quit.`	Terminate etb command loop or script, but stay in Prolog top level.
`regtest.`	Run built-in regression tests.
`script(` *<file name>* `[, step] ).`	Run the named command file, optionally pausing after each command.
`script(` *<file name>* `[, verbose] ).`	Run the named command file, optionally verbose.
`selftest.`	Run built-in self tests.
`time(` *<etb command>* `).`	Execute *<etb command>* and report time stats.
`time(` *<etb command>*`,` *<reps>* `).`	Execute *<etb command>* the number of times indicated by *<reps>* and report total time stats.
`traceoff.`	Turn off tracing.
`traceon.`	Turn on tracing.
`traceone.`	Turn on tracing for only the next command.
`version.`	Display the current version number and version description.
`versions.`	Display past and current version numbers with descriptions.

Table 2 shows the data and control commands available in the ETB tool.

Table 2 Data and control commands of the ETB tool

`ac_export( <base name>, <format> ).`	Export the current assurance case to named sub-directory in the CAP directory.
`attach_case( <assurance case id> ).`	Attach the identified assurance case in the CASE repository.
`detach_case.`	Detach the currently attached assurance case.
`etb_reset.`	Perform reset of ETB databases including dynamically created persistent objects.
`etb_reset( <domain>, <name> ).`	Reset ETB database item *<name>* in database *<domain>*.
`etb_server.`	Initiate the ETB Assurance Server for background operation.
`etb_server( <arg> ).`	Initiate the ETB Assurance Server for background operation with the specified argument.
`etbt.`	Run currently pre-configured default ETB test.
`etbt( <test id> ).`	Run the identified ETB test.
`etbt( <test id>, <mode> ).`	Run the identified test in *<mode>*.
`etbtests.`	Run all ETB tests.
`import( <spec type>, <file>, <sid> ).`	Import the *<spec type>* specification from *<file>* and store in the KB as *<sid>*.
`instantiate_pattern( <pattern name>, <pattern arguments list>, <assurance case id> ).`	Instantiate a pattern from the KB into the CASES repository with the given case id.
`instantiate_pattern_list( <list of pattern_name-arguments_list pairs>, <assurance case id> ).`	Instantiate a list of patterns from the KB into the CASE repository with the given case id.
`load_model_v( <model id>, <policy var>, <platform var>, <config var> ).`	Load system model from the KB and instantiate the provided variables.
`update.`	Update CASES and EVIDENCE to incorporate asynchronously updated evidence.

In the future it is expected that additional advanced user commands for development and diagnostics will be added.

2.1.3 Timing Commands

The ‘etb’ Assurance Tool has two commands for timing execution of other ‘etb’ commands. The first, time(etb_command) times the execution of a single command. The second, time(etb_command, repetitions) executes the command for the number of times specified by the repetitions argument, and reports timing statistics. The second version turns off console output from the repeated execution of the command to eliminate the substantial timing impact of generating console output, leaving a result that is more representative of the processing time that would be consumed by a server to do the equivalent work load.

2.1.4 Command procedures and scripts

There are predefined command procedures (“procs”) that run the examples and can be used for testing and demonstrations. At the “etb>” prompt a predefined procedure (e.g. named “myproc”) can be run with the command `proc(myproc).` It can be run with verbose output with the command `proc(myproc,verbose).` It can be made to prompt and wait for user instruction to proceed (empty line input) with the command `proc(myproc,step).`

The file procs.pl that defines the predefined procedures. The procedures utilise the same commands available at the command prompt. The user may define additional procedures in the procs.pl file for subsequent execution as above.

A sequence of ‘etb’ commands can also be stored in a file, in which case it is referred to as a *script*. Scripts may be run with a `script` command, analogous to the `proc` command, with the script file name substituted for the stored procedure name. `verbose` and `step` are valid options also for the execution of scripts.

2.2 ENHANCED ASSURANCE CASE PATTERN SPECIFICATION LANGUAGE

The external language representation is easily constructed from a graphical representation of a GSN assurance case and vice versa. The present implementation of the language does not yet include some of the standard GSN structural abstraction extensions for AC patterns but these extensions are part of the currently proposed new requirements for O-ETB.

2.2.1 Basic Syntax

Top level of assurance case pattern

- <ac pattern> ::= ac_pattern(<pattern id>, <formal arguments>, <goal>)
- <formal arguments> is a list
- <goal> ::= goal(<id>, <claim>, <context>, <body>)

- <context> ::=

x

2.2.2 Syntax of parameterized elements

2.2.2.1 Pattern Parameters

2.2.2.2 Extensions

The GSN Community Standard introduces a graphical notation for these features but this requirement suggests a textual representation that is consistent with the graphical notation and considered adequate for O-ETB in the context of MedSecurance. To that end, O-ETB will allow the following constructs in the target of GSN relations:

- alternative(<list of alternative elements>) – *optionality* – one or more (inclusive OR)
- alternative (<list of alternative elements>, <min>, <max>) – *optionality* – m of n, e.g. 1 of 3 (exclusive OR)
- optional(<element>) – *optionality* – 0 or 1 occurrence of the instantiated <element>
- multiple(<element>) – *multiplicity* – 1 or more occurrences of the instantiated <element>
- multiple (<element>, <cardinality>) – *multiplicity* – definite number of instantiated occurrences of <element>
- multiple (<element> , <low>, <high>) – *multiplicity* – indefinite number of instantiated occurrences of <element> ranging from <low> to <high>

These abstractions are applicable to all GSN relation types (i.e., SupportedBy and InContextOf).

There will also need to be some operations on "AC forms" (ACs that are at any stage of instantiation from uninstantiated pattern up to fully instantiated AC). Such operations include determining whether there are still uninstantiated terms, or finding uninstantiated terms, etc.

2.3 ASSURANCE SERVER

Note: This section describes features that are tentatively anticipated in a forthcoming O-ETB release but which are not yet committed.

2.3.1 Server sessions

To be able to service requests to work on multiple assurance cases simultaneously, each user process communicating with the server to create, update or delete an assurance case must establish as session with the server. The associated session identifier will be associated with an assurance case in the repository. Further API requests specifying the session identifier will be processed in the context of the assurance case associated with the session identifier.

2.3.2 Server APIs

A programmatic interface, in the form of RESTful APIs, constitutes the Assurance Server Interface. A specification of the proposed interface is forthcoming.

3. INSTALLATION AND OPERATION

3.1 OVERVIEW

The O-ETB system is composed of two components, the 'etb' Assurance Tool and the 'etb-server' Assurance Server. The Assurance Tool is used to develop and assurance case patterns and develop assurance cases. The ETB Assurance Tool and Server use those patterns to create instantiated assurance cases. These components share a common set of Prolog source modules and their separate executables are constructed using the `make` command with a simple ***makefile***, which accompanies the sources. A primary objective of the 'etb'/'etb-server' development is to create a lightweight and highly portable evidential tool bus framework that can be easily adapted to different situations and applications, that has a minimum of external dependencies, and that requires minimal resources to run. This objective has been achieved to a high degree in the current implementation.

3.2 PREREQUISITES

The 'etb' Assurance Tool and the 'etb-server' Assurance Server are implemented in the Prolog language and require the SWI-Prolog environment to run. This is the software's primary external dependency apart from the operating system. The version of SWI-Prolog used for the current version is version 8.2.4 available from www.swi-prolog.org.

SWI-Prolog is available for several operating environments, including Microsoft Windows (64 bit) and (32 bit), MacOS X 10.6 and later on Intel, and several Linux versions including Ubuntu. It is also available in Docker containers and as a source distribution that one can build locally.

Our O-ETB implementation uses *only* the libraries that come with the Prolog distribution or small open source utility packages that are included in the O-ETB repository. Furthermore, the functional architecture has been organized so as to place adaptation into the hands of application developers without requiring modification to the core implementation. Rather the implementation is adaptable through the contents of its O-ETB Knowledge Base.

The software is provided as a set of Prolog source files and/or as "executable" files that have the Prolog runtime environment already linked in. Prior to installing and building the software it is required that Prolog be installed. The exception to this requirement is if there is already a compiled version of 'etb' and 'etb-server' for the target platform, in which case all the dependencies are already linked into the executable.

3.3 INSTALLING AND RUNNING THE 'ETB' ASSURANCE TOOL

The etb assurance tool is implemented in Prolog and requires the SWI Prolog environment to run. The etb tool can be provided as a set of Prolog source files

and/or as an "executable" that has the Prolog runtime environment already bundled in. This executable is made by the shell script mketb, located with the source files that must be run in an environment that has SWI Prolog installed.

3.3.1 Install needed software

Install the following:

- SWI Prolog – Prolog language system and libraries – version 8.2.4 or later (http://www.swi-prolog.org)
- gsn2x – rendering a textual representation of GSN as a graphic image – not yet used but expected to be used (https://github.com/jonasthewolf/gsn2x)
- graphviz / dot – graphical rendering of textual dot notation (https://graphviz.org/download/)
- make – standard Unix environment build tool
- find – standard Unix environment tool

SWI Prolog is available for several operating environments, including Mac, Windows, and Linux.

The environment must be set up so that the Prolog runtime can exec the external software items listed above using "shell", "exec" or "forkexec".

3.3.2 Install the O-ETB source files and/or executable

The current version of the etb tool consists of a directory tree including source files and example files. The distribution is provided as a zip file of this directory tree or as a GitHub repository.

3.3.3 Compile the 'etb' policy tool

If a ready-made executable 'etb' has been provided it may be executed directly from a command shell prompt. If this is the case skip down to the next section.

Otherwise, in the source directory "src" execute the "make" command. It will use the provided makefile.

Note that the compiled 'etb' executable may be invoked with the command line options detailed in Section 2.1.1.

Alternatively, to compile manually, start SWI-Prolog from a command shell prompt using the name of the SWI-Prolog executable (usually 'swipl', 'swi-pl', or something similar, depending on how it was installed).

After printing a short banner SWI-Prolog will display its prompt "?- ".

At the Prolog prompt enter "[etb]." (without the quotes but with the period)

Prolog will compile the code and print “true.”

Execute the code by entering at the Prolog prompt “etb.”

Now you should see the ‘etb’ prompt “etb> ‘

3.3.4 Test the installed ‘etb’ tool

The O-ETB Assurance Tool has some self-tests built in. These should be run to ensure that everything is working correctly. Follow the instructions to run the ‘etb’ tool. If you compiled manually in Prolog start it with the Prolog prompt command “etb(self_test).” This will run some built-in self-tests when it starts. To not run the self-tests simply start the tool with the Prolog prompt command “etb.”

The self-tests can also be run by starting ‘etb’ normally and entering at the ‘etb’ prompt “selftest.”

Procedures made up of ‘etb’ commands may be predefined in the procs.pl file. Look at the ones there and try them by entering the etb command “proc(ProcName).”, where ProcName is the name of one of the procedures defined in procs.pl.

3.3.5 Running the examples

There are several examples included with the sources of the O-ETB Policy Tool. These include examples described in documents and PowerPoint slide decks used to introduce the O-ETB concepts.

There are predefined procedures (“procs”) that run the examples. At the etb> prompt a predefined procedure (e.g. named “myproc”) can be run with the command `proc(myproc).` It can be run with verbose output with the command `proc(myproc,verbose).`

It is instructive to read the file procs.pl that defines the predefined procedures. The procedures consist of the same commands available at the command prompt. The user may define additional procedures in the procs.pl file for subsequent execution as above.

3.4 INSTALLING AND RUNNING THE ‘ETB-SERVER’

The O-ETB Assurance Server etb-server is implemented in Prolog and requires the SWI-Prolog environment to run. The server can be provided as a set of Prolog source files and/or as an “executable” that has the Prolog runtime environment already bundled in. This executable is created by the makefile located with the source files and the make command must be run in an environment that has SWI Prolog installed.

3.4.1 Install SWI-Prolog

SWI Prolog is available for several operating environments, including Mac, Windows, and Linux. See http://www.swi-prolog.org.

3.4.2 Install the O-ETB source files and/or etb-server executable

The current version of O-ETB consists of a directory tree including source files, prototype structure for the runtime data files and example files. The distribution is provided as a repository on GitHub.

3.4.3 Initiating the 'etb-server'

The O-ETB server may started from the 'etb' policy tool. Though it may be useful for development and debugging, in normal use the assurance server should be started with the compiled executable 'etb-server'. This is preferable since it allows the command line options to be specified.

If you do want to start the server from the policy tool follow the instructions above to get 'etb' running. After starting 'etb' it offers the prompt "etb>". There are a set of basic commands available in the normal mode (admin) and an extended set of commands for use by a developer in development mode (advanced). Entering the command "help" will list the available commands in the current mode. If you want to load any policy files, do it now with the 'etb' command "import(policy(PolicyFileName)).", where PolicyFileName is the name of a .pl file relative to the execution directory. You can also combine policies with the 'etb' "compose" command. When you have the desired policies loaded and composed, start the server from the 'etb' tool using the command "server(PortNumber).", where PortNumber is an unused TCP port. The server will be started and will be listening to that port for calls to its RESTful API. A server started in this way will expect the default admin token (the string "admin_token", without the quotes) in the policy administration API calls.

The 'etb-server' can be run in the background from a startup script. To the command 'etb-server' add any desired command line options and append a "&" to run the process in the background.

3.4.4 Test the installed 'etb-server'

There are a number of shell scripts of curl commands included with the source in the TEST subdirectory. These scripts can be run to send a sequence of requests to the server to test for known correct answers. Basic tests are in `servercurltest.sh`. (Note: These scripts run illustrative tests, they do not run exhaustive tests.)

A top-level script `run-nn-tests.sh` runs all of the numbered test scripts. Expected results are contained in corresponding numbered files.

The etb tool can be easily extended in several ways.

- Commands can be added by modifying the `command` module to add a `syntax`, `semantics` (optional), `help`, and `do` clause for the new command. A `syntax` clause must be added for the command. This clause declares the command name and parameters, and what mode the command belongs to, admin or advanced. Admin commands are available in admin mode, but also accessible in the advanced mode but not vice versa.

- The self-test framework is implemented in the `test` module. Tests for specific new modules can be added in the `TEST` subdirectory. An example of a test definition file for the `etb` module is implemented in `TEST/etb_test.pl`.

- New predefined 'etb' command procedures can be added to the `procs` module. A `proc` clause is added for each new procedure to be defined. There are examples in the `procs.pl` file.

Global parameters are set in the `param` module. Settable parameters (those that can be changed from the 'etb' command line with the `set` command) are itemized in a list `settable_params`. For example, the parameter audit_logging is a settable parameter. Adding new settable parameters requires the new parameter name to be added to the list of settable parameters and to the `dynamic` directive above it in a fashion similar to the other entries.

3.5 O-ETB TOOL & SERVER DEPLOYMENT FOR MEDSECURANCE

To be supplied in collaboration with Unparallel Innovation.

4. PRINCIPLES OF OPERATION

4.1 ARGUMENT PATTERNS

4.2 ARGUMENT PATTERN INSTANTIATION

O-ETB workflow is driven primarily by assurance case argument pattern instantiation. Construction of the argumentation and the generation of the associated supporting evidence is accomplished by the instantiation of a chosen top-level pattern using information from a specified system model.

Error! Reference source not found. provides the definition of the pattern instantiation rewrite rules. Notational conventions used: $A \Rightarrow C$ represents a sequent where A represents the sentential form to be rewritten and C the resultant sentential form; $[\ Z\ ]_{M,\mu}$ denotes the instantiation of an assurance case element Z with respect to the model M and using the mapping of formal parameters μ. The given top-level pattern becomes the current sentential form to be instantiated. Instantiation proceeds by left-to-right recursive descent on the structure of the form, triggering specific evidence generating tool agents when evidence (leaves) are encountered.

$$[\![\ \underline{\text{pattern}}(\ Id, Args, Goal\ )\ ]\!]_{\mathcal{M},\mu} \Rightarrow [\![\ Goal\ ]\!]_{\mathcal{M},\mu/Args}$$

$$[\![\ \underline{\text{goal}}(Id, Claim, [\ Ctxt, \ldots\ ], [Support, \ldots]\ ]\!]_{\mathcal{M},\mu} \Rightarrow$$
$$\underline{\text{goal}}(Id\#\mu,\ [\![\ Claim\ ]\!]_{\mathcal{M},\mu}, [\ [\![\ Ctxt\ ]\!]_{\mathcal{M},\mu}, \ldots\ ], [\ [\![\ Support\ ]\!]_{\mathcal{M},\mu}, \ldots])$$

$$[\![\ \underline{\text{context}}(Claim)\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{context}}(\ [\![\ Claim\ ]\!]_{\mathcal{M},\mu}\ )$$
$$[\![\ \underline{\text{assumption}}(Claim)\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{assumption}}(\ [\![\ Claim\ ]\!]_{\mathcal{M},\mu}\ )$$
$$[\![\ \underline{\text{justification}}(Claim)\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{justification}}(\ [\![\ Claim\ ]\!]_{\mathcal{M},\mu}\ )$$

$$[\![\ \underline{\text{strategy}}(Claim, SArgs, [Ctxt, \ldots], [Goal, \ldots])\ ]\!]_{\mathcal{M},\mu} \Rightarrow$$
$$\underline{\text{strategy}}(\ [\![\ Claim\ ]\!]_{\mathcal{M},\mu}, [\ [\![\ Ctxt\ ]\!]_{\mathcal{M},\mu}, \ldots], [\bigcup_{\mu':SArgs\to\mathcal{M}} [\![\ Goal\ ]\!]_{\mathcal{M},\mu\sqcup\mu'}, \ldots\ ])$$

$$[\![\ \underline{\text{goal-ref}}(Id)\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{goal-ref}}(Id\#\mu)$$

$$[\![\ \underline{\text{pattern-ref}}(Id, \text{AArgs})\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{goal-ref}}(\ GId\#\ [\![\ \text{AArgs}\ ]\!]_{\mathcal{M},\mu}\ ),\ \textbf{eval}\ \ [\![\ Pattern\ ]\!]_{\mathcal{M},\mu}$$

(where $Pattern$ is the pattern identified by Id and GId is its top goal identifier)

$$[\![\ \underline{\text{evidence}}(Cat, Claim, [\ Ctxt, \ldots\ ]\ )\ ]\!]_{\mathcal{M},\mu} \Rightarrow$$
if call Agent@$Cat(\mathcal{M}, \mu)$ **== (<u>valid</u>,** $xref$**) then**
$\underline{\text{evidence}}(\ [\![\ \text{Claim}\ ]\!]_{\mathcal{M},\mu}, [\ [\![\ \text{Ctxt}\ ]\!], \ldots], xref)$
else $\perp$

Figure 3 Pattern instantiation rewrite rules

4.3 CASES REPOSITORY

4.4 EVIDENCE REPOSITORY

5. ANNEX A: POPULATING THE O-ETB KNOWLEDGE BASE

5.1 MODEL INFORMATION

5.2 ARGUMENT PATTERNS

5.3 WORKFLOW INFORMATION

6. ANNEX B: O-ETB TESTING

The functional testing of the Open Evidential Tool Bus (O-ETB), is accomplished with a combination of two mechanisms:

- a built-in self-test feature, accessible with the `selftest` command of the 'etb' Assurance Tool, that exercises key functions through internal interfaces, and,
- shell scripts that exercise the RESTful APIs to the same functions.

Both of these methods are used as regression tests during ongoing development and maintenance of O-ETB.

Beyond functional correctness, tests will be developed for volume and load, performance and endurance, and correctness tests for non-functional support as described below.

6.1 VOLUME AND LOAD TESTS

For volume and load tests a simple framework of scripts was constructed. The top level script `stress-top` first invokes a `stress-etbapi` script that calls the Assurance Server's interface to establish a session to be used for subsequent assurance case and evidence API tests and then initiates multiple parallel copies of a script that repeatedly makes a series of API calls. The queries made by this script comprise ….
The `stress-top` script takes a numeric argument that it passes along to each of the sub-scripts to tell them how many iterations of the test cases to perform. The value used for this count should be sufficient to assure that the several sub-scripts are running in parallel. The sub-script checks the results of each query, according to the expected result. Any failure of these comparisons is reported as an "ERROR" output along with the failing test case identifier.

6.2 PERFORMANCE AND ENDURANCE TESTS

To facilitate performance tests, a `time` command is provided by the 'etb' Policy Tool to allow the user to time the execution of any single 'etb' command, or to time multiple executions of a single command or a command script. In the case that multiple executions are requested, text output from the repeated command executions are sent to a null stream, so as not to overly bias the timing result for the execution of the core functions by the accumulated latency of file or terminal I/O resulting from working through the interactive command interpreter.

This `time` command is used to measure the time to perform multiple executions of a representative Assurance Tool command:

```
instantiate( … ).
```

This query is by far the most frequently occurring work of the O-ETB system, others being one-time or infrequent calls, which are not only less frequent but much less computationally demanding. Consequently, this test approach is deemed to provide adequate practical coverage.

correctness of the AC instantiation function. Therefore, a series of tests has been developed to demonstrate this correctness, at the command level, at the HTTP API level, and as self-tests perfomed at startup or on demand. The self-tests are constructed within a test framework organized by source module that facilitates addition of tests for new modules and extension of existing module tests. The correctness test framework compares execution results to known answers for the test cases and report discrepancies.

The 'etb' Assurance Tool has built-in self-tests. These may be run to ensure that expected results are produced after O-ETB installation, or as a regression test during further code development. The self-tests can be run by starting 'etb' normally and entering at the "O-ETB>" prompt the command "selftest" or "regtest", which run all of the currently configured tests.

Self-tests and regression tests use the test framework implemented by the module `test.pl`. Tests for any module may be created in a similar fashion to the currently implemented tests, by creating a `TEST/mod_test.pl` file for a corresponding module file `mod.pl`, which should have an include directive for its corresponding test file. Tests defined as startup tests are run by the selftest command and those defined as regression tests will be run in addition to the defined self-tests by the regtest command.

The file TEST/instantiate_test.pl defines self-test cases with expected results for the pattern instantiation functions in the code file `instantiate.pl`. A representative sample of currently configured test cases are shown in Table 3.

Table 3: Instantiation correctness self-tests

Test case ID	Test case	Expected result
tc01		Succeed
tc02		Succeed
tc03		Succeed
tc04		Fail
tc05		Fail

6.3 HTTP API CORRECTNESS TESTS

The HTTP APIs are tested using an environment-provided utility command (`curl`) that enables HTTP requests to be constructed and sent to a network port on the local or a remote machine (the current tests use a fixed reserved port on the local host). The tests are packaged as shell script files in the TEST/nn-tests directory, with each file name beginning with a number. The directory also contains for each such script a file of the expected result in JSON format. A master script, `run-nn-tests.sh`, runs each of the individual numbered test scripts in turn and checks the current results against expected results.

Table 4 identifies the automated HTTP API tests currently defined. Every such test is represented by a shell script file with the name corresponding to the Test case ID, and an expected result files (e.g. 01-expected-json) and therefore, as the expected results can be rather lengthy, the expected result is not given here in the table.

Table 4: O-ETB correctness HTTP API tests

Test case ID	Test case description